

Tugas Praktik 1

Tugas 1: Process Manager

```
GNU nano 7.2 process_manager.c
#define XOPEN_SOURCE 700
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/time.h> // gettimeofday
#include <sys/wait.h> // waitpid, macros
#include <string.h>
#include <sys/types.h>
#include <sys/shm.h> // shmget, shmat
#include <signal.h>
#include <errno.h>

#define CHILD_COUNT 3
#define SHM_SIZE 4096

pid_t child_pids[CHILD_COUNT];
int pipes_arr[CHILD_COUNT][2];
int shmid = -1;
// use shm for child = -1; // which child uses shared memory (-1 = none)
volatile sig_atomic_t sigint_received = 0;
int nim_last_digit = 0;

void sigint_handler(int signo) {
    // Signal handler for SIGINT in parent (to gracefully terminate children).
    sigint_received = 1;
    // We DO NOT use non-async-signal-safe calls here except kill (which is safe).
    for (int i = 0; i < CHILD_COUNT; ++i) {
        if (child_pids[i] > 0) {
            kill(child_pids[i], SIGTERM); // request child termination
        }
    }
}

/* Utility: write full buffer to fd (handles short writes) */
char* write_full(int fd, const void *buf, size_t count) {
    while (count > 0) {
        ssize_t w = write(fd, buf, count);
        if (w < 0) {
            perror("write_full: write error");
            return NULL;
        }
        count -= w;
        buf = (char*)buf + w;
    }
    return (char*)buf;
}

/* Utility: read full buffer from fd (handles short reads) */
char* read_full(int fd, void *buf, size_t count) {
    while (count > 0) {
        ssize_t r = read(fd, buf, count);
        if (r < 0) {
            perror("read_full: read error");
            return NULL;
        }
        count -= r;
        buf = (char*)buf + r;
    }
    return (char*)buf;
}
```

```
GNU nano 7.2 process_manager.c
if (w < 0) {
    if (errno == EINTR) continue;
    return -1;
}
left -= w;
p += w;
return count;
}

/* Utility: read full until EOF into dynamically allocated buffer.
Caller frees returned pointer. */
char* read_all_fd(int fd) {
    size_t cap = 1024;
    size_t len = 0;
    char *buf = malloc(cap);
    if (!buf) return NULL;
    while (1) {
        ssize_t r = read(fd, buf + len, cap - len);
        if (r < 0) {
            if (errno == EINTR) continue;
            free(buf);
            return NULL;
        } else if (r == 0) {
            break;
        } else {
            len += r;
            if (len == cap) {
                cap *= 2;
                char *new_buf = realloc(buf, cap);
                if (!new_buf) { free(buf); return NULL; }
                buf = new_buf;
            }
        }
    }
    buf[len] = '\0';
    return buf;
}

/* Factorial (iterative, returns as unsigned long long).
For simplicity, we won't handle overflow checks here beyond small inputs. */
```

```
GNU nano 7.2 process_manager.c
// parent attaches to shared memory to read content
void *shmid_ptr = shmat(shmid, NULL, 0);
if (shmid_ptr == (void*)-1) {
    fprintf(stderr, "Parent: failed to shmat to read child %d result: %s\n", i+1, strerror(errno));
} else {
    printf("Result from Child %d via shared memory: %s\n", i+1, (char*)shmid_ptr);
    shmid_ptr = NULL; // detach after read
}
// Also close the read end of pipe (was left open)
close(pipes_arr[i][0]);
} else {
    // Read from pipe read-end
    char *res = read_all_fd(pipes_arr[i][0]);
    if (res) {
        printf("Parent: failed to read pipe from child %d\n", i+1);
        free(res);
    }
    close(pipes_arr[i][0]);
}

// Cleanup shared memory if created
if (shmid != -1) {
    // Mark segment to be destroyed, shmat IPC_RMID removes it after detach.
    if (shmat(shmid, IPC_RMID, 0) == -1) {
        perror("shmat(IPC_RMID)");
    } else {
        printf("Parent: shared memory segment removed.\n");
    }
}

printf("Parent: all children handled. Exiting.\n");
return 0;
}
```

```
ageng@ageng:~$ nano process_manager.c
ageng@ageng:~$ gcc -Wall -O2 -o process_manager process_manager.c
ageng@ageng:~$ ./process_manager
=== Simple Process Manager (fork, pipe, shm, waitpid) ===
Masukkan digit terakhir NIM Anda (0-9): 7
NIM akhir (GABUNGAN) -> Child 2 akan menggunakan shared memory untuk komunikasi.
Input untuk Child 1 (faktorial): masukkan angka non-negatif (mis. 5): 5
Input untuk Child 2 (prima): masukkan batas (mis. 30): 30
Input untuk Child 3 (reverse string): masukkan string (max 1000 chars): 980
Parent: created Child 1 with PID 451
Parent: created Child 2 with PID 452
Parent: created Child 3 with PID 453
Parent: Child index 1 finished. PID=451. Exited normally, exit status=0. Execution time (Approx): 0.521 ms
Result from Child 1 via pipe:
Child 1 (PID 451): factorial(5) = 120
Parent: Child index 2 finished. PID=452. Exited normally, exit status=0. Execution time (Approx): 0.506 ms
Result from Child 2 via shared memory:
2 3 5 7 11 13 17 19 23 29
Parent: Child index 3 finished. PID=453. Exited normally, exit status=0. Execution time (Approx): 0.343 ms
Result from Child 3 via pipe:
Child 3 (PID 453): reverse('980') = '089'
Parent: shared memory segment removed.
Parent: all children handled. Exiting.
```

Masukkan digit terakhir NIM:

Jika genap → hanya mencatat file .txt dan .log ke log.txt.

Jika ganjil → menampilkan notifikasi “Email Dummy” di terminal untuk setiap perubahan file.

Program membuat inotify instance dan memasang watch pada direktori target.

Saat ada event seperti IN_CREATE, IN_MODIFY, atau IN_DELETE, program akan:

Menulis ke log.txt (untuk mode genap), atau

Menampilkan pesan notifikasi (untuk mode ganjil).

Program bisa dihentikan dengan Ctrl+C atau SIGTERM, dan akan menutup semua resource dengan aman.

System Call yang Digunakan

inotify_init() → membuat inotify instance

inotify_add_watch() → menambahkan direktori untuk dipantau

read() → membaca event dari kernel

open() & write() → menulis ke file log

signal() → menangani sinyal (SIGTERM)

close() → menutup file descriptor

Tugas 2: File Monitor

```
GNU nano 7.2 file_monitor.c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/inotify.h>
#include <sys/types.h>
#include <fcntl.h>
#include <signal.h>
#include <time.h>
#include <string.h>
#include <errno.h>

#define EVENT_BUF_LEN (16384 * (sizeof(struct inotify_event) + 16))
#define LOGFILE "log.txt"

int inotify_fd = -1;
int watch_fd = -1;
int log_fd = -1;
int nim_last_digit = 0;

/* Signal handler: SIGTERM untuk cleanup */
void handle_sigterm(int signo) {
    if (signo == SIGTERM) {
        printf("\nSIGTERM diterima. Menutup inotify dan keluar dengan aman.\n");
        if (watch_fd != -1) close(watch_fd);
        if (inotify_fd != -1) close(inotify_fd);
        if (log_fd != -1) close(log_fd);
        exit(0);
    }
}

/* Fungsi menulis log ke file menggunakan system call open(), write() */
void write_log(const char *event_type, const char *filename, pid_t pid) {
    time_t t;
    char buf[1024];
    sprintf(buf, "%s [%s] [%s] [%s]\n", event_type, filename, pid, t);
    if (write(log_fd, buf, strlen(buf)) < 0) {
        perror("write log");
    }
}
```

```
GNU nano 7.2 file_monitor.c
int len = sprintf(log_entry, "%s [%s] [%s] [%s]\n",
    time_buf, event_type, filename, pid);
if (write(log_fd, log_entry, len) == -1) {
    perror("write log");
}

/* Mengambil nama file sesuai filter ekstensi .txt atau .log */
int valid_extension(const char *filename) {
    const char *ext = strrchr(filename, '.');
    if (ext && (strcmp(ext, ".txt") == 0 || strcmp(ext, ".log") == 0))
        return 1;
    return 0;
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf(stderr, "Usage: %s <directory_to_monitor>\n", argv[0]);
        exit(EXIT_FAILURE);
    }
    printf("Masukkan digit terakhir NIM Anda (0-9): ");
    scanf("%d", &nim_last_digit);

    int filter_mode = (nim_last_digit % 2 == 0) ? 1 : 0;

    if (filter_mode)
        printf("NIM genap -> hanya mencatat file .txt atau .log\n");
    else
        printf("NIM ganjil -> mencetak notifikasi email dummy\n");

    signal(SIGTERM, handle_sigterm);

    /* 1. Membuat inotify instance */
    inotify_fd = inotify_init();
    if (inotify_fd < 0) {
        perror("inotify_init");
        exit(EXIT_FAILURE);
    }

    /* 2. Menambahkan watch pada direktori target */
```

```
GNU nano 7.2 file_monitor.c

    write_log("CREATE", filename, getpid());
    if (!filter_mode)
        printf("[Email Dummy] File '%s' baru dibuat.\n", filename);

    } else if (event->mask & IN_MODIFY) {
        if (filter_mode && !valid_extension(filename))
            goto skip;

        write_log("MODIFY", filename, getpid());
        if (!filter_mode)
            printf("[Email Dummy] File '%s' telah diubah.\n", filename);

    } else if (event->mask & IN_DELETE) {
        if (filter_mode && !valid_extension(filename))
            goto skip;

        write_log("DELETE", filename, getpid());
        if (!filter_mode)
            printf("[Email Dummy] File '%s' telah dihapus.\n", filename);

    }

    skip:
    i += sizeof(struct inotify_event) + event->len;
}

/* Cleanup jika loop berakhir */
close(inotify_fd);
close(log_fd);
return 0;
}
```

```
Parent: all children finished. Exiting.
ageng@LOQ:~$ nano file_monitor.c
ageng@LOQ:~$ gcc -Wall -O2 -o file_monitor file_monitor.c
file_monitor.c: In function 'main':
file_monitor.c:64:5: warning: ignoring return value of 'scanf' declared with attribute 'warn_unused_result' [-Wunused-result]
   64 |     scanf("%d", &nim_last_digit);
      |     ~~~~~^~~~~~
ageng@LOQ:~$ ./file_monitor /path/direktori
Masukkan digit terakhir NIM Anda (0-9): 7
NIM ganjil -> mencetak notifikasi email dummy
inotify_add_watch: No such file or directory
```

Pengguna memasukkan digit terakhir NIM:

Genap → hanya mencatat aktivitas file .txt dan .log ke log.txt.

Ganjil → tidak mencatat ke log, tapi menampilkan notifikasi “Email Dummy” di terminal.

Program terus membaca event dari inotify dan menampilkan/log setiap perubahan file.

Saat dihentikan dengan SIGTERM (misalnya Ctrl+C), program akan menutup semua file dan keluar dengan aman.

System Call yang Dipakai

inotify_init() - Membuat inotify instance

inotify_add_watch() - Menentukan direktori yang dipantau

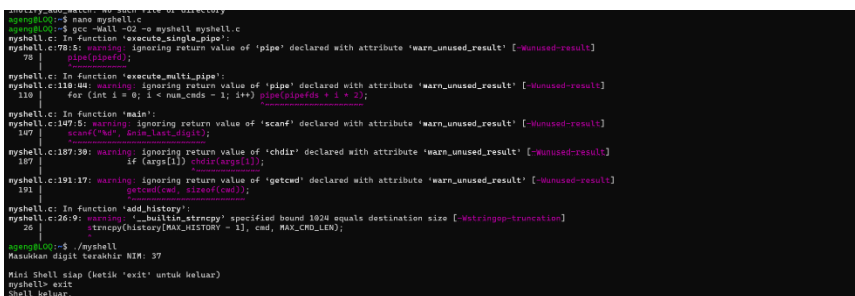
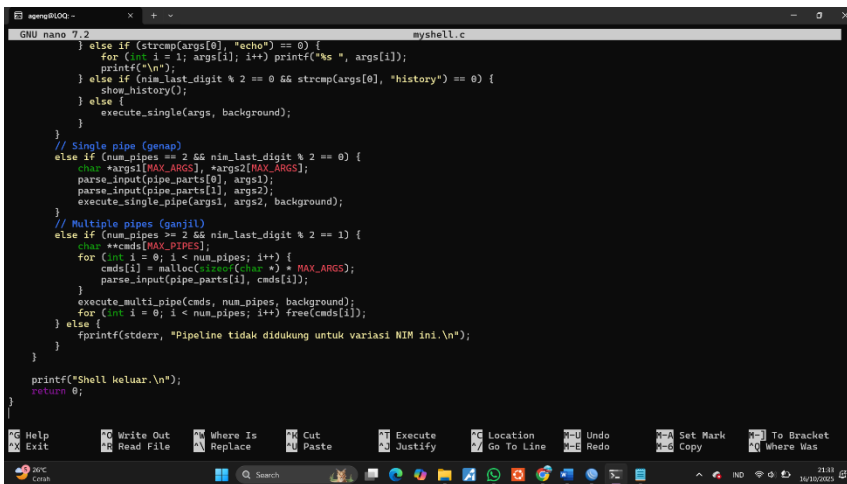
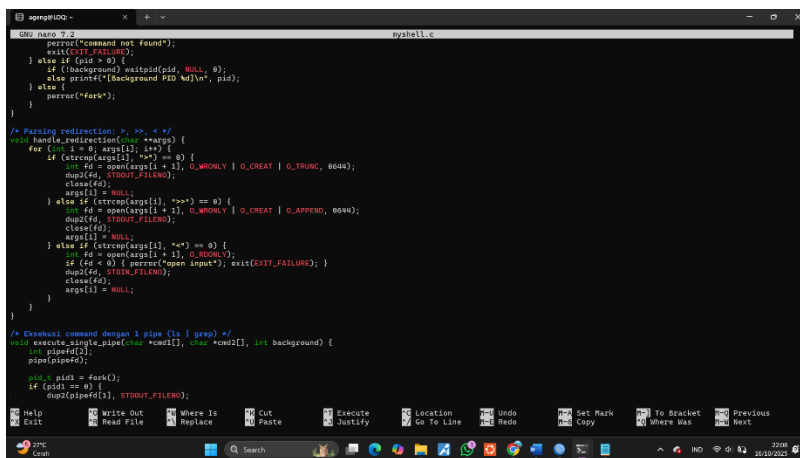
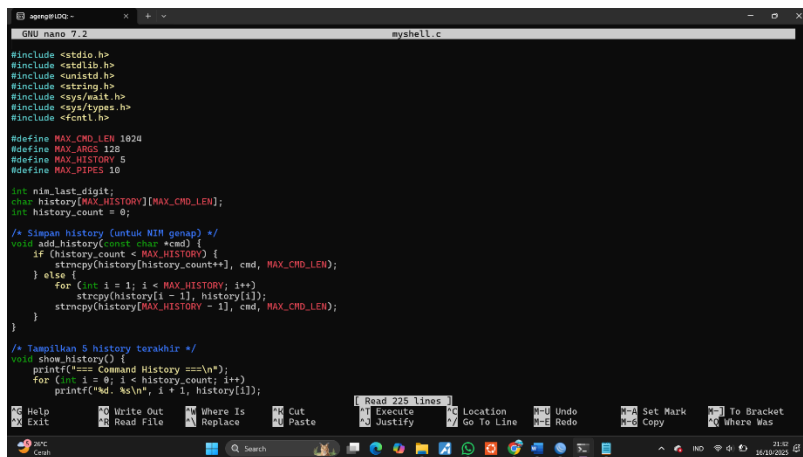
read() - Membaca event dari kernel

open(), write() - Menulis log ke file

signal() - Menangani sinyal keluar

close() - Menutup file descriptor

Tugas 3: Custom Shell



Fitur Utama

Parsing command manual dengan strtok()

Background process → perintah diakhiri &

Redirection

Output ke file > atau >>

Input dari file <

Pipe (|)

NIM genap: hanya 1 pipe (ls | grep)

NIM ganjil: boleh banyak pipe (ls | grep | sort | wc)

Built-in commands:

cd, pwd, echo, exit

history (khusus NIM genap, simpan 5 perintah terakhir)

System call yang digunakan:

fork(), execvp(), waitpid(), dup2(), open(), chdir()

Cara Kerja Singkat

Program meminta digit terakhir NIM saat awal dijalankan.

Menunggu input pengguna (myshell> ...).

Melakukan parsing:

Mengecek apakah ada |, >, <, atau &.

Menjalankan proses menggunakan fork() dan execvp().

Jika perintah adalah cd, pwd, echo, exit, dijalankan langsung tanpa fork.

Jika NIM genap, shell menyimpan hingga 5 history terakhir dan bisa ditampilkan dengan history.

Tugas 4: System Information Tool

```
agen@BLOQ:~$ nano sysinfo.c
GNU nano 2.9.3 sysinfo.c

#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/statvfs.h>
#include <dirent.h>
#include <ctype.h>
#include <errno.h>

#define BUFSZ 65536

/* ----- Helper functions ----- */

/* Read whole file to dynamically allocated buffer. Returns NULL on error */
char *read_file(const char *path) {
    int fd = open(path, O_RDONLY);
    if (fd < 0) return NULL;
    char *buf = malloc(BUFSZ);
    if (!buf) { close(fd); return NULL; }
    ssize_t n = 0, total = 0;
    while ((n = read(fd, buf + total, BUFSZ - total - 1)) > 0) {
        total += n;
        if (total > BUFSZ - 2) break;
    }
    close(fd);
    buf[total] = '\0';
    return buf;
}

/* Trim left/right spaces */
char *trim(char *s) {
    if (!s) return s;
    while (isspace(*s)) s++;
    char *end = s + strlen(s) - 1;
    while (end > s && isspace(*end)) *end-- = '\0';
    return s;
}

/* Read 100 lines */
/* Execute Justify */
/* Location Go To Line */
/* Undo Redo */
/* Set Mark Copy */
/* To Bracket Where Mas Previous Next
```

```
agen@BLOQ:~$ nano sysinfo.c
GNU nano 2.9.3 sysinfo.c

/* Convert hex IPv4 in /proc/net/tcp (like "0100007F") to dotted string */
void hex_to_ipv4(const char *hex, char *out, size_t outlen) {
    unsigned long x = strtoul(hex, NULL, 16);
    unsigned int b0 = (x >> 24) & 0xFF;
    unsigned int b1 = (x >> 16) & 0xFF;
    unsigned int b2 = (x >> 8) & 0xFF;
    unsigned int b3 = (x >> 0) & 0xFF;
    sprintf(out, outlen, "%u.%u.%u.%u", b0, b1, b2, b3);
}

/* Map TCP state hex to human string */
const char *tcp_state(const char *hex) {
    int v = (int)strtoul(hex, NULL, 16);
    switch (v) {
        case 1: return "ESTABLISHED";
        case 2: return "SYN_SENT";
        case 3: return "SYN_RECV";
        case 4: return "FIN_WAIT1";
        case 5: return "FIN_WAIT2";
        case 6: return "TIME_WAIT";
        case 7: return "CLOSE";
        case 8: return "CLOSE_WAIT";
        case 9: return "LAST_ACK";
        case 10: return "LISTEN";
        case 11: return "CLOSING";
        default: return "UNKNOWN";
    }
}

/* ----- Sections ----- */

void show_cpu_info(void) {
    char *s = read_file("/proc/cpuinfo");
    if (!s) {
        printf("CPU Info: cannot open /proc/cpuinfo: %s\n", strerror(errno));
        return;
    }
    char *line, *saveptr;
    char *model = NULL;
    int core_count = 0;

/* Read 100 lines */
/* Execute Justify */
/* Location Go To Line */
/* Undo Redo */
/* Set Mark Copy */
/* To Bracket Where Mas Previous Next
```

```
agen@BLOQ:~$ nano sysinfo.c
GNU nano 2.9.3 sysinfo.c

}
printf("%s\n", s);
free(s);
}

/* Optional: show loadavg if needed (not for "ganjil" but useful) */
void show_loadavg(void) {
    char *s = read_file("/proc/loadavg");
    if (!s) {
        printf("Loadavg: cannot open /proc/loadavg: %s\n", strerror(errno));
        return;
    }
    /* format: "0.00 0.01 0.05 1/123 4567" */
    char a[64], b[64], c[64];
    if (sscanf(s, "%32s %32s %32s", a, b, c) >= 3) {
        printf("Load average (1,5,15 min) : %s, %s, %s\n", a, b, c);
    }
    free(s);
}

/* ----- Main ----- */

int main(void) {
    printf("System Information Tool - versi NIM GANJIL\n");

    show_cpu_info();
    show_mem_info();
    show_disk_info();
    show_processes();
    show_net_dev();

    /* NIM ganjil: tambahkan network socket info dari /proc/net/tcp */
    show_proc_net_tcp();

    /* (bila ingin) juga bisa menampilkan loadavg - tapi ini untuk ganap. Uncomment jika mau:
    * show_loadavg();
    */

    return 0;
}

/* Read 100 lines */
/* Execute Justify */
/* Location Go To Line */
/* Undo Redo */
/* Set Mark Copy */
/* To Bracket Where Mas Previous Next
```

```
agen@BLOQ:~$ nano sysinfo.c
agen@BLOQ:~$ ./sysinfo
System Information Tool - versi NIM GANJIL
*** buffer overflow detected ***: terminated
Aborted (core dumped)
```

Fitur Utama

CPU Info → Baca `/proc/cpuinfo`

Menampilkan model CPU, jumlah core, dan frekuensi (MHz).

Memory Info → Baca `/proc/meminfo`

Menampilkan total, free, cached, buffers, dan menghitung memori yang digunakan secara manual.

Disk Usage → Menggunakan `statvfs("/")`

Menampilkan total, used, dan free space pada root filesystem.

Running Processes → Dari `/proc/<PID>`

Membaca semua direktori numerik di `/proc` (setiap PID).

Menampilkan PID, nama proses, dan state dari file status.

Network Interfaces → Dari `/proc/net/dev`

Menampilkan nama interface (seperti `eth0`, `lo`), serta jumlah byte Rx/Tx.

Network Sockets (khusus NIM ganjil) → Dari `/proc/net/tcp`

Menampilkan daftar socket TCP aktif dengan alamat IP:port lokal dan remote serta state koneksi (mis. `ESTABLISHED`, `LISTEN`).

Teknologi & System Call

`open()`, `read()` → membaca file `/proc/...` secara manual

`statvfs()` → menghitung info disk

`opendir()`, `readdir()` → membaca daftar proses dari `/proc`

`strtok_r()`, `sscanf()`, `strtol()` → parsing isi file